

PRODUCT BACKLOG

Project: Fábula Dental

Management System for a Pediatric Dental Clinic

Team: CodeXplorers | Version: 4.0 | Date: June 11th, 2026

Technologies (Unit 1 — Frontend): PHP · MongoDB · Bootstrap 5 · JavaScript · Docker

Technologies (Unit 2 — Backend APIs): Node.js · TypeScript · Express · Prisma · PostgreSQL (Supabase)

Team Members:

- Andrés Cárdenas
- Chuqui Kerly
- Díaz Victoria

1. Project Description

Fábula Dental is a web-based clinical management system specializing in pediatric dentistry. The platform allows users to manage patients, track payments, and manage dental supply inventory, with differentiated access based on user role (Administrator, Dentist, Receptionist).

In Unit 2, the project evolved from a monolithic PHP/MongoDB architecture to a distributed two-server TypeScript REST API system, maintaining the original frontend as a legacy reference while building a production-grade backend.

Technology Stack — Unit 1 (Legacy Frontend, archived in old-frontend/)

- Backend: PHP 8.x with MVC architecture
- Database: MongoDB Atlas (collections: patients, payments, supplies)
- UI: Bootstrap 5.3, Vanilla JavaScript, HTML5, CSS3
- Infrastructure: Docker (Dockerfile)
- Authentication: PHP sessions with role control

Technology Stack — Unit 2 (Current Backend Architecture)

- CRUD Server (port 3000) @ 34.58.145.37: Node.js + Express + TypeScript + Prisma ORM
- Business Logic Server (port 3001) @ 35.254.153.60: Node.js + Express + TypeScript + fetch API
- Database: PostgreSQL via Supabase (tables: patients, payments, supplies)
- ORM: Prisma with pg adapter (PrismaPg + pgbouncer connection pooling)
- Deployment: GCP — CRUD: 34.58.145.37 | Business Logic: 35.254.153.60

Architecture Overview

The system uses two decoupled servers:

- CRUD Server: Handles all direct database operations (CREATE, READ, UPDATE, DELETE) using Prisma to query PostgreSQL.
- Business Logic Server: Consumes the CRUD server via HTTP fetch calls, applies domain calculations (payment status, supply expiration, financial analysis, patient clinical logic) and returns enriched responses.

System Actors

- Administrator: Manages dental supply inventory and views financial analytics
- Dentist: Registers and manages patient information
- Receptionist: Registers and manages patient payments
- Visitor: Consults public information on treatments

2. User Stories

User stories describe the functionalities of the system from the perspective of each actor, following the standard agile format: As [role], I want [action], for [benefit].

Note: US-06 and US-07 were previously merged in a single malformed row. US-09 and US-10 had "System" as the actor — corrected to Receptionist and Administrator respectively, as user stories require a human business role. Corrected rows are highlighted.

No change		Corrected row (v4.0)		
ID	As...	Wanna...	For...	Acceptance Criteria
US-01	System user	Log in with username and password	Access the module corresponding to my role	- Redirection based on role after successful login - Error message for invalid credentials - Fields cannot be empty
US-02	Dentist	Register a new patient with ID, name, DOB, and reason for consultation	Have the complete medical history in the system	- ID card with exactly 10 digits mandatory (HTTP 400 if violated) - legalRepresentative field required if patient < 18 years (HTTP 400 if missing) - Successful registration saves to PostgreSQL via CRUD API and returns 201
US-03	Dentist	View, edit, and delete patients from the list	Keep patient information up-to-date	- Table loads patients from GET /fabuladental/patients - Online editing sends PUT to the CRUD API - Deletion sends DELETE and removes row without reloading
US-04	Receptionist	Record a payment indicating patient, amount, type, and method	Keep track of the clinic's income and payments	- Payment type: Deposit or Final - Final → Completed; Deposit → Partial - Amount > 0; date not future; Patient ID = 10 digits (HTTP 400 on any violation) - Method: Cash, Card, or Transfer
US-05	Receptionist	View the complete payment history with its current status	Have financial traceability	- All payments via GET /fabuladental/payments (BL Server) - Colors: Completed = green, Partial = yellow, Pending = red - Records can be edited and deleted
US-06	Administrator	Register dental supplies with name, quantity, cost, and dates	Control the clinic's inventory	- Quantity min. 1 unit (HTTP 400 if < 1) - Expiration date >= order date (HTTP 400 if violated) - Missing required fields return HTTP 400 - Auto status: Current / NextExpiration / Expired
US-07	Administrator	View inventory with alerts for products nearing expiration	Anticipate replenishment needs	'NextExpiration' if expiration date <= 30 days from today 'Expired' if date has passed - Differentiated visual indicator in table
US-08	Clinic visitor	View the list of available treatments without logging in	Know about services before an appointment	- Accessible from login header - Shows name, description, post-treatment recommendations - No authentication required
US-09	Receptionist	Retrieve payment history with calculated billing status	Provide financial traceability per patient	- GET /fabuladental/payments → derived status list (BL Server) - GET /fabuladental/payments/:paymentId → specific payment record

ID	As...	Wanna...	For...	Acceptance Criteria
US-10	Administrator	Perform financial analysis on inventory	Support administrative decision-making	• GET /fabuladental/payments/patients/:patientId → full patient history • GET /fabuladental/supplies/asset-valuations → total inventory value (BL Server) • GET /fabuladental/supplies/restock-provisions → consolidated restock budget • GET /fabuladental/supplies/expiration-losses → total loss from expired items • GET /fabuladental/supplies/capital-risks → at-risk capital (NextExpiration items)

3. Product Backlog

The Product Backlog is ordered by epics and priority. Story points follow the Fibonacci scale (1, 2, 3, 5, 8).

High Priority	Average Priority	Low Priority	Done	Pending	Planned
---------------	------------------	--------------	------	---------	---------

3.1 Functional Backlog Items

Items PB-01 to PB-21 carry over from v3.0 (with corrections noted). Items PB-26 to PB-30 are new entries added in v4.0 to reflect implemented Business Logic endpoints that were previously untracked.

ID	User Story	Description	Acceptance Criteria	Priority	Pts	State
PB-01	Role-based login	System user, I want to log in with my credentials to access features of my specific role.	- Validates username/password - Redirects to correct view by role (Admin, Dentist, Receptionist) - Error if credentials incorrect	High	5	Done
PB-02	Role-based access control	System administrator, I want each user to only have access to their corresponding modules.	- Admin → Inventory only - Dentist → Patients only - Receptionist → Payments only	High	3	Done
PB-03	Log out	Authenticated user, I want to securely log out to protect the clinic's information.	'Log Out' button destroys the PHP session and redirects to login	High	2	Done
PB-04	New patient registration	Dentist, I want to register a new patient with their personal details.	- Captures: ID (10 digits), full name, DOB, phone, reason for consultation - legalRepresentative field required if patient < 18 (HTTP 400 if missing for minor) - Stored via POST /fabuladental/patients (CRUD Server). Returns HTTP 201	High	8	Done
PB-05	List of patients	Dentist, I want to see all registered patients.	- Table via GET /fabuladental/patients - Each record can be edited or deleted	High	5	Done
PB-06	Patient data editing	Dentist, I want to edit an existing patient's data.	'Save' sends PUT /fabuladental/patients/:patientId (CRUD Server) - Changes reflected in real time	High	5	Done
PB-07	Patient removal	Dentist, I want to delete a patient record.	- DELETE /fabuladental/patients/:patientId (CRUD Server) - Record disappears without page reload	Average	3	Done
PB-08	Legal rep validation for minors	Dentist, I want the system to require a legal representative for patients under 18.	- Calculates age from DOB. Under 18 without guardian → HTTP 400 error - Exposed via POST /fabuladental/patients/validate-legal-representative (BL Server)	High	3	Done
PB-09	Payment record	Receptionist, I want to record a patient payment with type, method, and amount.	- Captures: Patient ID (10 digits), date, amount (> 0), type (Deposit/Final), method (Cash/Transfer/Card) - HTTP 400 if ID ≠ 10 digits, amount ≤ 0, or date is in the future	High	8	Done

ID	User Story	Description	Acceptance Criteria	Priority	Pts	State
			Status auto-calculated. Stored via POST /fabuladental/payments (CRUD Server)			
PB-10	Payment history	Receptionist, I want to see all recorded payments.	- Shows payments with status (Pending/Partial/Completed) via GET /fabuladental/payments (BL Server) - Records editable and deletable	High	5	Done
PB-11	Payment edition	Receptionist, I want to edit a recorded payment.	- Editable fields. Sends PUT /fabuladental/payments/:paymentId (CRUD Server) - Changes reflected immediately	Average	3	Done
PB-12	Visual indicator of payment status	Receptionist, I want to see payment status with distinctive colors.	- Pending = red/orange - Partial = yellow - Completed = green	Average	2	Pending
PB-13	Supply record	Administrator, I want to register a new dental supply.	- Captures: name, quantity (min. 1), unit cost, order date, expiration date - Expiration date must be $\geq$ order date (HTTP 400 if violated) - HTTP 400 if quantity < 1 or required fields are missing - Stored via POST /fabuladental/supply (CRUD Server)	High	8	Done
PB-14	Inventory list with status	Administrator, I want to see the supply inventory with expiration status.	- Shows name, quantity, cost, dates - Status updated via POST /fabuladental/supplies/status-calculations (BL Server) - Statuses: Current, NextExpiration (≤ 30 days), Expired - GET /fabuladental/supplies/quantity-thresholds/:maxQuantity (CRUD) for low-stock filter - GET /fabuladental/supplies/statuses/:statusValue (CRUD) for status filter	High	5	Done
PB-15	Product expiration alert	Administrator, I want automatic identification of supplies about to expire.	Calculates expiration diff. If ≤ 30 days $\rightarrow$ NextExpiration with visual alert indicator	High	3	Done
PB-16	Editing and deleting supplies	Administrator, I want to edit or delete supplies.	- Edit: PUT /fabuladental/supplies/:id (CRUD Server) - Delete: DELETE /fabuladental/supplies/:id - List updates without page reload	Average	3	Done
PB-17	Treatments page	User, I want to see available treatments without logging in.	- Accessible from login page - Shows table: name, description, post-treatment recommendations - No authentication required	Average	3	Done
PB-18	Patient search and filtering	Dentist, I want to search for patients by name or ID.	- Real-time search field filtering the table by name and ID number - Pending API route definition in URI spec	Average	5	Pending
PB-19	Dashboard with statistics	Administrator, I want summarized metrics on the dashboard.	- Cards with dynamic counters - Data from API in real time - Pending API route definition in URI spec	Low	8	Pending

ID	User Story	Description	Acceptance Criteria	Priority	Pts	State
PB-20	Secure database authentication	Administrator, I want encrypted user credentials in the database.	- Credentials stored in PostgreSQL with bcrypt hashing - Login queries DB instead of hardcoded values	High	8	Pending
PB-21	Exporting reports	Receptionist/admin, I want to export payment and inventory lists.	- Export button in payment and inventory lists - Generates downloadable PDF/Excel file - Pending API route definition in URI spec	Low	8	Planned
PB-26	BL — Legal Representative Validation (API)	Dentist, I want the Business Logic server to validate legal representative presence for underage patients via API.	- POST /fabuladental/patients/validate-legal-representative (BL Server) - Returns { valid, requiresLegalRepresentative, message } - HTTP 400 if birthday format invalid or minor missing representative - HTTP 500 on internal error	High	2	Done
PB-27	BL — Birthday Countdown	Dentist, I want to know how many days are left until a patient's next birthday.	- POST /fabuladental/patients/calculate-days-to-birthday (BL Server) - Returns { daysUntilBirthday, isBirthdayWeek } - isBirthdayWeek = true if birthday is within 7 days - HTTP 400 if birthday format invalid	Low	2	
PB-28	BL — Senior Discount	Receptionist, I want to determine if a patient qualifies for a senior discount.	- POST /fabuladental/patients/calculate-senior-discount (BL Server) - Returns { isEligibleForDiscount, suggestedDiscountFactor } - Eligible if patient age >= 65 years - HTTP 400 if birthday format invalid	Average	2	
PB-29	BL — Consultation Time Estimate	Dentist, I want an estimated consultation time based on patient age and reason for consultation.	- POST /fabuladental/patients/estimate-consultation-time (BL Server) - Base time + extra minutes if age < 5, age > 70, or complex consultation reason - Returns { estimatedConsultationMinutes, baseMinutes } - HTTP 400 if birthday format invalid	Average	3	Done
PB-30	BL — Contact Priority	Receptionist, I want a contact priority score for a patient based on their contact details and urgency.	- POST /fabuladental/patients/calculate-contact-priority (BL Server) - Returns { contactPriorityScore, requiresImmediateAttention } - Score boosted by urgent keywords: dolor, urgencia, sangrado, emergencia - HTTP 500 on internal error	Average	3	

3.2 Technical Enablers

The following items were originally documented as User Stories PB-22 to PB-25 with "As the dev team..." framing. This is methodologically incorrect: User Stories must always be written from the perspective of a human business role receiving tangible value. Technical migration tasks are classified as Technical Enablers since the end user does not directly perceive a change in the graphical interface. They are relabeled TE-01 to TE-04.

ID	Technical Enabler	Description	Implementation Criteria	State
TE-01	TS Migration — Payments CRUD API (ex-PB-22)	Migrate the payments module from PHP/MongoDB to TypeScript/PostgreSQL on the CRUD Server.	- paymentRoutes.ts in CRUD server (port 3000) - POST, PUT, DELETE, GET /fabuladental/payments endpoints - Validation: 10-digit patientId, amount > 0, non-future date, valid type and method - Status auto-calculated. Zero TypeScript errors (tsc --noEmit)	Done
TE-02	TS Migration — Payments Business Logic API (ex-PB-23)	Expose payment business logic endpoints on the Business Logic Server.	- paymentsBusinessController.ts + paymentsBusinessRoutes.ts - GET /fabuladental/payments (history with derived status) - GET /fabuladental/payments/:paymentId (specific record) - GET /fabuladental/payments/patients/:patientId (patient history) - Consumes CRUD via fetch. Zero TS errors	Done
TE-03	TS Migration — Supplies Business Logic API (ex-PB-24)	Build financial analysis endpoints for inventory on the Business Logic Server.	- suppliesBusinessController.ts deployed and functional - POST /fabuladental/supplies/status-calculations - GET /fabuladental/supplies/asset-valuations - GET /fabuladental/supplies/restock-provisions - GET /fabuladental/supplies/expiration-losses - GET /fabuladental/supplies/capital-risks. Zero TS errors	Done
TE-04	TS Migration — Patients CRUD API (ex-PB-25)	Migrate the patients module to TypeScript/Prisma on the CRUD Server.	- patientRoutes.ts in CRUD server (port 3000) - POST, GET, PUT, DELETE /fabuladental/patients endpoints - Validates 10-digit ID, legalRepresentative for minors, birthday format - Prisma ORM + BigInt IDs. Zero TS errors	Done

Total story points: 144 pts | Completed (Sprints 1–4): 113 pts | Pending/Planned (Sprints 5–6): 31 pts

4. Sprint Planning

The table below summarizes the distribution of Backlog items in development Sprints, with the objective of each iteration and the current progress status.

Sprint 4 updated to include new patient Business Logic items (PB-26–PB-30) and to reference Technical Enablers (TE-01–TE-04) instead of the old PB-22–PB-25 user story framing.

Sprint	Items Included	Sprint Objective	Pts	State
Sprint 1	PB-01, PB-02, PB-03, PB-04, PB-08, PB-17	Functional authentication, patient registration with validations, and public treatment page.	19	Done
Sprint 2	PB-05, PB-06, PB-07, PB-09, PB-10, PB-13, PB-14, PB-15	Complete CRUD for patients, payment registration, inventory management with expiration alerts.	37	Done
Sprint 3	PB-11, PB-12, PB-16	Payment and inventory refinement: editing, deletion, and visual status indicators. (Note: PB-12 still pending.)	8	Done
Sprint 4 (Unit 2)	TE-01, TE-02, TE-03, TE-04, PB-26, PB-27, PB-28, PB-29, PB-30	Migration of all modules to TypeScript/PostgreSQL. Two-server architecture: CRUD (34.58.145.37:3000) and Business Logic (35.254.153.60:3001). Patient BL endpoints deployed on GCP.	41	Done
Sprint 5	PB-12, PB-18, PB-19, PB-20	Security: DB-stored credentials (bcrypt). Patient search. Statistical dashboard. Pending visual status indicator (PB-12).	23	Planned
Sprint 6	PB-21	Export of reports in PDF/Excel for payments and inventory.	8	Planned

5. Technical and Non-Functional Requirements

5.1 Implemented Functional Requirements

- RF-01: Authentication system with differentiated roles (Admin, Dentist, Receptionist)
- RF-02: Complete CRUD of patients with Ecuadorian ID validation (10 digits) — TypeScript + Prisma
- RF-03: Validation of legal representative for patients under 18 years of age
- RF-04: Complete CRUD of payments with types (Deposit/Final) and methods (Cash/Transfer/Card) — TypeScript + Prisma
- RF-05: Automatic derivation of payment status (Deposit → Partial, Final → Completed) in Business Logic layer
- RF-06: Complete CRUD of supplies with automatic expiration status calculation — TypeScript + Prisma
- RF-07: Automatic alerts for products with an expiration date ≤ 30 days (NextExpiration status)
- RF-08: Public treatment page accessible without authentication
- RF-09: RESTful APIs for each module (GET, POST, PUT, DELETE) — TypeScript/Express
- RF-10: Business Logic endpoints for financial analysis: asset valuation, restock provisioning, expiration losses, capital risk
- RF-11: Payment history endpoint with calculated status per payment record
- RF-12: Patient payment history endpoint filtered by patient ID
- RF-13: Legal representative validation endpoint for underage patients (PB-26)
- RF-14: Patient Business Logic endpoints — birthday countdown, senior discount, consultation time estimate, contact priority (PB-27–PB-30)

5.2 Non-Functional Requirements

- RNF-01 Security: PHP session control by role (legacy). Environment variables for DB credentials (.env, excluded from git). PB-20 pending bcrypt implementation.
- RNF-02 Usability: Responsive interface with Bootstrap 5. Compatible with mobile and desktop.
- RNF-03 Performance: CRUD operations do not require full page reload (fetch API). BL communicates with CRUD via HTTP.
- RNF-04 Scalability: PostgreSQL (Supabase) with pgbouncer connection pooling for horizontal scaling.
- RNF-05 Portability: Deployable on GCP or any Node.js-compatible environment.
- RNF-06 Maintainability: Clean separation between CRUD and Business Logic servers. TypeScript strict mode enforced.
- RNF-07 Validation: Backend input validation — 10-digit IDs, positive amounts, non-future dates, valid enums, inventory business rules.
- RNF-08 Type Safety: All backend code compiled with TypeScript strict mode with zero type errors.

6. Definition of Done (DoD)

A backlog item is considered DONE when it meets all of the following criteria:

- The code is implemented and working in the development/production environment
- The acceptance criteria have been verified manually (Postman or browser)
- The functionality has been tested with real data in PostgreSQL (Supabase)
- There are no console errors or unhandled exceptions
- For TypeScript modules: compiles with zero errors using `tsc --noEmit`
- The code follows the project's architecture: `routes/` and `controllers/` under each server's `src/`
- The functionality has been reviewed and approved by the CodeXplorers team